

# What Magentra’s prompts are missing, measured against Claude Code

A re-measurement of Magentra’s registered prompts against the leaked Claude Code system prompts, after the removal of four unshipped subsystems. The headline finding of the previous review has not improved — it has got worse, and now for a clearer reason.

|               |                                                                 |
|---------------|-----------------------------------------------------------------|
| PREPARED FOR  | M Ali Ozturk — Magentra                                         |
| DATE          | 30 July 2026                                                    |
| SUBJECT BUILD | Magenta 0.12.1.0 — 75 registered prompts, 27 tools (26 offered) |
| REFERENCE     | claude-code-opus-5, -fable-5, -opus-4.6                         |
| SOURCE        | system_prompts_leaks-main.zip                                   |
| SUPERSEDES    | Prompt Architecture Review, 28 July 2026 (Magenta 0.11.1.0)     |

## 1 Executive summary

Since the previous review Magentra has removed four subsystems that were registered but never shipped: the hireable crew, CAREFUL MODE, deterministic retrieval, and standing missions. That removed **five prompts and two tools** from every request. It did not fix the architectural finding — it isolated it.

FIXED COST SENT ON EVERY REQUEST (TOKENS)

| BUILD                               | INSTRUCTIONS | TOOL DESCRIPTIONS | TOTAL         | SPLIT            |
|-------------------------------------|--------------|-------------------|---------------|------------------|
| Claude Code · Opus 5                | 2,975        | 20,503            | 23,478        | 13% / 87%        |
| Claude Code · Fable 5               | 3,295        | 20,500            | 23,795        | 14% / 86%        |
| Claude Code · Opus 4.6              | 8,185        | 24,190            | 32,375        | 25% / 75%        |
| Magenta 0.11.1 (previous review)    | 4,500        | 4,462             | 8,962         | 50% / 50%        |
| <b>Magenta 0.12.1 (this review)</b> | <b>6,210</b> | <b>3,821</b>      | <b>10,031</b> | <b>62% / 38%</b> |

**The ratio moved the wrong way.** Instructions grew 38% while tool descriptions shrank 14%. Magentra now spends nearly two-thirds of its fixed budget telling the model how to behave, where the reference spends seven-eighths describing what its tools do. Every reference build sits between 13% and 25% instructions. Magentra is at 62%.

**Why this is the finding that matters.** Instruction text is advice the model may or may not follow. A tool description is read at the moment of choosing that tool, and it constrains the call being made. Guidance moved from the first into the second converts a hope into a constraint — and costs nothing extra, because the tool description is already being sent.

## The four findings, ranked by impact

| AREA          | FINDING                                                                                                       |
|---------------|---------------------------------------------------------------------------------------------------------------|
| Architecture  | 62% of fixed cost is instructions vs. 13% in the reference; the gap widened after the removals (§3).          |
| Tool surface  | 26 tool descriptions average 147 tokens. The reference averages roughly 790 (§4).                             |
| Concentration | Six system prompts carry 61% of all instruction tokens; two carry 27% (§5).                                   |
| Hygiene       | Registry and product are now in step: zero prompts for removed features, one tool deliberately disabled (§6). |

## 2 Method and caveats

Magenta’s column is measured directly from the built engine: the prompt registry is enumerated with `promptCatalog()`, each entry resolved through `promptText()`, and counted with the engine’s own `estimateTokens()`. Tool descriptions are the prompts under the `tool.` namespace. The numbers are therefore what the product actually sends, not an estimate from source files.

**Caveat, stated plainly.** The Claude Code column is carried forward unchanged from the 28 July review. The reference corpus has not changed, but its instruction/tool split was derived by that review’s extraction method, which this one did not re-derive. The two columns are therefore comparable in magnitude and ratio, not to the token. Nothing in the findings below turns on a difference smaller than several hundred tokens.

Counts exclude conditional sections that are absent on a typical turn (project memory, atlas header, standards header) and everything sent only inside a subagent.

### 3 Finding 1 — The ratio moved the wrong way

#### ARCHITECTURE

The previous review found Magentra at a 50/50 split and recommended moving guidance into tool descriptions. Four subsystems were removed since. Their tools went with them — `CrewRun` and `BackpackSearch`, 365 tokens — and so did their prompts. What remained was the instruction layer, which grew.

| LAYER                       | 0.11.1       | 0.12.1        | CHANGE               |
|-----------------------------|--------------|---------------|----------------------|
| Instructions (48 prompts)   | 4,500        | 6,210         | +1,710 (+38%)        |
| Tool descriptions (26 sent) | 4,462        | 3,821         | −641 (−14%)          |
| <b>Fixed total</b>          | <b>8,962</b> | <b>10,031</b> | <b>+1,069 (+12%)</b> |

The removals were correct — they deleted code that never ran. But they cut the cheaper half of the budget. The result is a build that sends *more* fixed context than before while offering the model *less* information about its tools.

### 4 Finding 2 — Tool descriptions are thin

#### TOOL SURFACE

Magenta registers 27 tools and offers 26. Those 26 descriptions total 3,821 tokens — an average of 147 each. The reference builds spend roughly 20,500 tokens across a comparable tool count, averaging near 790.

| MAGENTRA TOOL                              | TOKENS | WHAT THE DESCRIPTION OMITTS                                                                                                                                    |
|--------------------------------------------|--------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>Bash</code>                          | 534    | The reference devotes several thousand tokens to this one tool: quoting rules, when to prefer a dedicated tool, banned patterns, working-directory discipline. |
| <code>Monitor</code>                       | 338    | —                                                                                                                                                              |
| <code>GraphQLQuery</code>                  | 314    | —                                                                                                                                                              |
| <code>Read</code>                          | 278    | No guidance on offset/limit for large files, which is where the token cost actually lands.                                                                     |
| <code>Agent</code>                         | 238    | No guidance on when delegation beats doing the work inline — the decision that governs subagent cost.                                                          |
| <code>Edit</code>                          | 124    | No worked example of a failing match, the most common error mode.                                                                                              |
| Remaining 20 tools: 2,105 tokens combined. |        |                                                                                                                                                                |

This is where the instruction budget should go. Rules currently stated once in `system.code-style` or `system.working-method` — where they apply to every turn whether relevant or not — would bind harder inside the description of the tool they govern.

### 5 Finding 3 — Instruction cost is concentrated

## CONCENTRATION

Six prompts carry 3,806 of the 6,210 instruction tokens — 61%. Two carry 1,653 between them, 27% of the instruction layer, on every request.

| PROMPT                     | TOKENS       | SHARE OF INSTRUCTIONS |
|----------------------------|--------------|-----------------------|
| system.code-style          | 904          | 14.6%                 |
| system.communication       | 749          | 12.1%                 |
| finishing.runtime-evidence | 579          | 9.3%                  |
| system.harness             | 566          | 9.1%                  |
| system.working-method      | 558          | 9.0%                  |
| system.autonomy            | 530          | 8.5%                  |
| <b>Six prompts</b>         | <b>3,806</b> | <b>61.3%</b>          |

system.code-style is the clearest candidate to redistribute. Most of what it says is about writing and editing files; it would apply more reliably inside Write and Edit, which are read precisely when the model is about to write or edit.

**Not a call to shorten.** The reference sends far more total context than Magentra does — 23,478 tokens against 10,031. The problem is not volume; it is placement. Magentra can afford to grow its tool layer substantially and still send less than half what Claude Code sends.

## 6 Finding 4 — Registry hygiene is now clean

### HYGIENE

Every prompt in the registry belongs to a shipping feature. This was not true at the last review, and the difference is worth recording:

- **Zero prompts for removed features.** The crew, backpack, CAREFUL MODE, retrieval and mission prompts are gone from the registry along with their code.
- **Zero orphan prompt ids.** Every id the prompt lab references resolves against the live registry, and every live prompt appears in the lab — verified in both directions.
- **Workflow is disabled deliberately.** Its description is emptied, so the registry withholds it: 27 tools registered, 26 offered. This was the largest single tool description in the previous review at 604 tokens.

The prompt lab is therefore an accurate console again: what it lists is what the product sends.

## 7 Where Magentra is ahead

Two capabilities in the reference corpus have no equivalent, and they are worth protecting:

- **A live prompt registry.** Every piece of model-facing prose is addressable by id, editable without a rebuild, overridable per workspace, and disableable by emptying it. The leaked builds have no such seam — their prompts are compiled in.

- **Deterministic end-of-turn rungs.** The runtime-evidence floor and the self-verify check are enforced in code against what the turn actually did — which files it changed, whether it ever executed anything — not by asking the model to self-assess. The reference relies on instruction text for the same outcome.

## 8 Recommended order of work

| # | ACTION                                                                                                                                                      | EST. EFFECT             |
|---|-------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------|
| 1 | Move the file-editing half of <code>system.code-style</code> into the <code>Write</code> and <code>Edit</code> descriptions.                                | −450 instr<br>+450 tool |
| 2 | Expand <code>Bash</code> toward the reference’s depth: quoting, when to prefer <code>Grep</code> / <code>Glob</code> / <code>Read</code> , banned patterns. | +600 tool               |
| 3 | Give <code>Agent</code> an explicit delegate-vs-inline rule; give <code>Read</code> an offset/limit rule for large files.                                   | +300 tool               |
| 4 | Re-measure. Target a split near 35% / 65% — still instruction-heavier than the reference, but no longer inverted.                                           | —                       |

Steps 1–3 are net-neutral to slightly positive on total tokens and move guidance to where it binds. None of them requires an engine change — all four are edits in the prompt lab.